Modular Continual Learning

Guillem Ramírez, Piotr Nawrot and Xiaotang Du

Abstract

Continual learning is essential for real-world systems when there is a need to handle a stream of different tasks efficiently. Existing work on continual learning either focuses on overcoming catastrophic forgetting or promoting knowledge sharing, with little success in addressing these two issues together. We propose a novel modular architecture for continual learning that takes advantage from adapters. As the model is sequentially trained on a stream of tasks, we dynamically introduce trainable modules to represent task-specific knowledge and compose them by soft routing to share knowledge across tasks. Our approach provides a guarantee for preventing catastrophic forgetting by allocating dedicated parameters for each task and an explicit mechanism to enable knowledge transfer by routing. Our experiments on GLUE and the CrossFit benchmark suggest that this approach can effectively avoid forgetting while encouraging forward transfer of knowledge to learn later tasks.

1 Introduction and Motivation

Many real-world systems need to learn dynamically from a stream of input data, such as conversational agents (Lee, 2017; Mazumder et al., 2019), medical image analysis (Lenga et al., 2020) and recommendation systems (Yuan et al., 2021). Although human beings can continually acquire knowledge and apply previous knowledge to understand new concepts, machine learning models often rely on fixed knowledge and suffer from the catastrophic forgetting problem when they are adapted to new domains. How to enable these systems to transfer knowledge across tasks without forgetting previously learned knowledge remains a challenging open problem.

Continual learning (CL) studies the problem of sequentially training the model on a set of different tasks with the goal of achieving two main objectives: **preventing catastrophic forgetting** and **encouraging knowledge transfer**. These properties are particularly important for systems in real-world scenarios. Providing a guarantee for no catastrophic forgetting ensures the system performs effectively and robustly on previously learned tasks. On the other hand, in order to address low-resource tasks, the system should be able to leverage knowledge from relevant tasks. Additionally, CL system brings the unique benefit of significantly reducing the computational cost and memory assumption since we no longer need to train different models from scratch for each downstream task.

Most existing work in CL focuses on developing approaches for overcoming catastrophic forgetting based on regularisation (Kirkpatrick et al., 2017; Schwarz et al., 2018), parameter isolation (Rusu et al., 2016; Mallya and Lazebnik, 2018) and memory replay (Shin et al., 2017; Rolnick et al., 2019). Although some previous papers attempt to address both issues, their methods are shown to suffer from catastrophic forgetting when the training tasks requires substantially different skills (Ke et al., 2021).

To develop CL models that can address both aspects, we draw inspiration from recent advances in parameter efficient fine tuning. As the pre-trained language models (PLM) becomes increasingly larger (Treviso et al., 2022), fine-tuning all the parameters of such models on downstream tasks leads to undesirable properties of ending up with a completely different model for each task. Some recent research focuses on developing approaches for effectively tuning PLMs with fewer trainable parameters, known as *adapters*, while achieving comparable or better performance than fine-tuning (Houlsby et al., 2019; Li and Liang, 2021; Hu et al., 2021; Mao et al., 2021).

Previous work successfully applies parame-

ter efficient approach to represent and combine knowledge across tasks in transfer learning or multitask learning settings by taking advantage of the modularity and composability of adapters (Pfeiffer et al., 2021; Ponti et al., 2022). Similar to parameter isolation methods, such approach can allocate dedicated parameters to learn task-specific representations, which provides a guarantee to prevent forgetting.

In this paper, we propose a new method for continual learning based on modular representations learned by parameter efficient fine-tuning, **modular continual learning (MCL)**. This method guarantees by design that the performance on seen tasks is not affected by new ones, and at the same time it benefits from an inductive bias. Specifically, as the model is sequentially trained on a set of different tasks, we introduce a Low-rank Adapter (Hu et al., 2021) for each task to capture task-specific knowledge. To enable forward transfer of knowledge while training the latest adapter, we interpolate the representations learned by previous adapters and that of current adapter via soft routing. Our experiments on GLUE and CrossFit benchmark show that this approach can effectively leverage previous knowledge to learn later tasks during training.

2 Method

2.1 Continual learning

The continual learning setting assumes we are given a sequence of tasks $\mathcal{T} = (t_1, \ldots, t_{|\mathcal{T}|})$, where each task consists of a dataset $\mathcal{D}_i = \{(x_1, y_1), \ldots, (x_n, y_n)\}$. We are only given one task at a time and we cannot revisit examples from previous tasks. We set the following objectives:

- **Forward transfer**: improve performance on a newly seen task by implicitly conditioning on previous tasks.
- **Catastrophic forgetting**: avoid degraded performance on previously seen tasks (French, 1999).

2.2 Adapters

With increasing model sizes, recent work has focused on parameter efficient fine-tuning, which we go over in the *Related Work* section.

One of such methods is called Adapters, which are small modules that are added to the original model during fine-tuning on a task. The model's weights remain unchanged while these task-specific modules are fine-tuned. First proposed in computer vision (Rebuffi et al., 2017), it is soon demonstrated their potential in NLP, showing similar performance to full fine-tuning on the GLUE benchmark with dramatically decreased trainable parameters (Houlsby et al., 2019).

LoRA Hu et al. (2021) introduces Low-Rank Adaptation (LoRA). Given a parameter matrix $W \in \mathbb{R}^{d\times d}$ corresponding with the linear projection of the query, key, value or output of the self-attention layer, we define two new matrices A and B that provide a low rank approximation: $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{r\times d}$, with $r << d$; A is randomly initialised and B is initialised with zeros. W is frozen and the new layer output is a combination of the new and old output: $y = (W + \beta A \times B)x$, where $\beta \in \mathbb{R}$ is a hyperparameter.

LoRA is explicitly designed for the Transformer architecture (Hu et al., 2021). Liu et al. (2022) shows that LoRA is the most parameter-efficient Adapter for few-shot settings.

2.3 Soft routing

Algorithm 1: Modular Continual Learning with Soft Routing

Freeze θ
$\Phi \leftarrow \{\}$
for $t \in \mathcal{T}$ **do**
  $\phi_t \sim \mathcal{N}(0, \sigma^2 I)$
  $\Phi \leftarrow \Phi \cup \phi_t$
  $\alpha \sim \mathcal{U}(\frac{1}{t})$
  while not converged **do**
    $\boldsymbol{x}, \boldsymbol{y} \sim \mathcal{D}_t$
    $\alpha' \leftarrow \text{softmax}(\alpha)$
    $\theta' \leftarrow \theta + \sum_{i=1}^{|\Phi|} \alpha'_i \, \phi_i$
    $\phi_t, \alpha \leftarrow [\phi_t, \alpha] - \eta \nabla_{\phi_t, \alpha} \ell(f(\boldsymbol{x}; \theta'))$
  $\phi_t \leftarrow \sum_{i=1}^{|\Phi|} \alpha'_i \, \phi_i$
  Freeze ϕ_t

We propose a novel method to train adapters in a continual learning setting. In particular, given a new task, we use soft routing to leverage the adapters trained on previous tasks as a parametrisation of the new adapter that we learn. This enables forward transfer by explicitly using information learned from previous tasks that is stored in corresponding LoRAs' parameters. Furthermore,

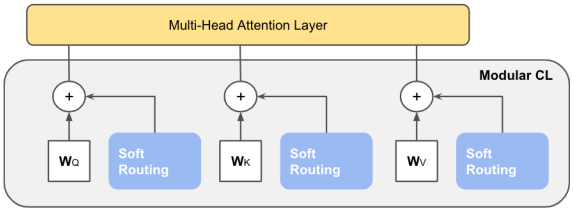


Figure 1: Overview of the model architecture for modular continual learning. In each Transformer layer, we introduce Low-rank adapter for each task and combine the learned representations by soft routing. Details of soft routing module are shown in Figure 2.

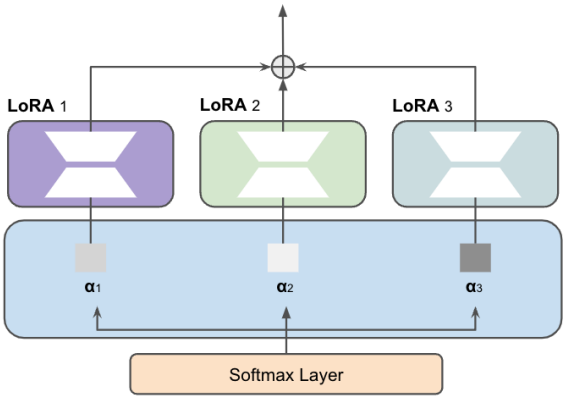


Figure 2: Illustration of soft routing. Given a vector parameter α, we use softmax or alternative function to convert it to a probabilistic distribution, which can be treated as the weights for interpolating different Low-rank adapters. We then compute a weighted average of these representations.

we store all previously trained adapters so that they can be easily accessed, which prevents the catastrophic forgetting by design.

Algorithm 1 contains the details of our method. We start from a model with parameters θ, which we freeze. We can adapt this model with any adapter $\phi \in \Phi$. We observe a sequence of tasks $t_1, \ldots, t_{|\mathcal{T}|}$ from an inventory $\mathcal{T}$. Given the i-th task t_i, we initialise a new adapter ϕ_i and a vector $\alpha \in \mathbb{R}^i$, which are the only learnable parameters for that task. We then apply a normalization function such as softmax on α, and use the output to weight previous (frozen) adapters and the current one.

We experiment with different initialisations of α and with functions alternative to softmax (lines 6 and 9 of Algorithm 1). We build our architecture upon LoRAs because of their proven robustness (Hu et al., 2021). Since LoRAs are linear, our method is equivalent to having a new LoRA adapter, which we obtain through interpolating previous ones and current one according to their weights.

Our method takes inspiration from Polytropon (Ponti et al., 2022; Caccia et al., 2022), which learns a set of skilled adapters during a pre-training step and then combines them to learn a new task. However, we move from multi-task learning setting to continual learning, where we need to enable forward transfer through a sequence of tasks. Different from Polytropon, we have one-to-one correspondence between tasks and skills since we introduce a new adapter for each new task.

3 Experimental Setup

3.1 Datasets

Currently there are no existing benchmarks for the evaluation of continual learning approaches in NLP field. Therefore, we build our work upon some standard multi-task learning datasets and conduct experiments by training the model sequentially on a subset of selected tasks.

GLUE The General Language Understanding Evaluation (GLUE) benchmark (Wang et al., 2018) consists of a collection of sentence or sentence-pair language understanding tasks. In our experiments, we consider four types of tasks in GLUE. **Linguistic acceptability** (CoLA) captures whether a given sentence is grammatical or not. **Sentiment classification** (SST-2) requires the model to assign binary sentiment labels to given sentence. **Sentence similarity** (MRPC, STS-B, QQP) measures whether two sentences have similar semantic content. **Natural language inference** (QNLI, RTE) requires the model to categorize the relationship between a premise sentence and a hypothesis sentence. We assess the model performance on CoLA by Matthews correlation coefficient and evaluate the performance on STS-B by Pearson coefficient respectively. For remaining tasks, we use either accuracy or F1-score as evaluation metrics, following Wang et al. (2018).

CrossFit CrossFit challenge is designed to evaluate cross-task generalisation ability and to encourage research in building general purpose few-

		Baselines						Soft Routing			
Task	Metric	Private		Shared		EWC		Softmax		Sigmoid	
COLA	Matthews-Corr	38.42	±6.1	38.42	±6.1	38.42	±6.1	38.42	±6.1	38.42	±6.1
MRPC	F1	73.41	±1.5	74.65	±1.8	74.10	±1.7	73.52	±2.1	73.87	±1.4
QNLI	Accuracy	77.59	±1.3	75.71	±1.3	75.80	±1.0	77.50	±0.7	77.86	±0.9
QQP	F1	73.81	±2.0	74.13	±1.8	74.47	±1.8	73.86	±2.4	74.31	±2.6
RTE	Accuracy	62.93	±1.6	62.43	±1.7	62.79	±1.7	63.31	±2.2	63.69	±2.2
SST2	Accuracy	85.64	±0.9	83.78	±1.2	83.63	±1.1	85.08	±0.8	85.54	±0.9
STSB	Pearson Corr	81.47	±0.9	80.72	±1.3	81.10	±1.5	82.08	±0.9	82.39	±0.9
Average	——	70.47	±0.8	69.98	±0.8	70.04	±0.8	70.54	±0.8	70.87	±0.8

Table 1: Results on the test datasets of the GLUE benchmark. We evaluate the model on each task right after the model is trained on it (no catastrophic forgetting). We report the average and standard deviation of the metrics generated with ten different random seeds.

shot learners. The benchmark consists of 160 few-shot NLP tasks, covering a wide range of task types, including text classification, regression, question answering, dialogue generation and summarization etc. These tasks are partitioned into three non-overlapping sets T_{train}, T_{dev} and T_{test} for training, development and testing respectively. Each task is formulated in a united text-to-text format. There are 5 different training and development splits that are sampled from the original training set and a test split created from original development set. The training/development split has 16 samples per class for classification and regression tasks, and for other tasks, the number of instances per training/development split is 32.

In our experiments, we only evaluate our approaches on the tasks in T_{test}. For each dataset, we process the training and development data by merging five different shards into a single dataset. Since different tasks have different metrics, we consider a collection of six metrics for evaluation, i.e. classification F1, accuracy, QA F1, Rouge-L, Matthew correlation and exact match (EM), that covers all relevant tasks.

3.2 Models

For GLUE tasks we use BERT$_{base}$ (Devlin et al., 2018) , following Houlsby et al. (2019) and subsequent work (Mao et al., 2021). We use T5$_{large}$ (Raffel et al., 2019) for CrossFit as it is the model that inspires framing tasks in text-to-text fashion (Ye et al., 2021). We download the weights of pre-trained models and corresponding tokenizers from HuggingFace (Wolf et al., 2019). We do not fine-tune any of the pre-trained weights or change the standard text preprocessing pipeline. Instead, we only add LoRA (Hu et al., 2021) linear projections and train them to optimise the task objective. Trainable parameters in LoRA amount to 0.18% and 1.22% of total model parameters for BERT$_{base}$ and T5$_{large}$, respectively. These parameters can be trained in various ways, which may prevent catastrophic forgetting (Private Adapter), enable forward transfer (Shared Adapter), or account for both (Soft Routing). We also compare with Elastic Weight Consolidation (Kirkpatrick et al., 2017), an established baseline for continual learning, to further validate our approach.

3.2.1 Baselines

To investigate whether our approach can encourage forward transfer while preventing catastrophic forgetting, we compare the performance of our model with three baselines.

Private Adapter For each new task, we fine-tune a new adapter. Therefore, this baseline does not suffer from catastrophic forgetting and it does not allow for forward transfer.

Shared Adapter We sequentially fine-tune the same adapter for all the tasks. That is, after training the adapter on $n-1$ previous tasks, we fine-tune it on the n-th task, and use these weights for the next task.

This baseline is expected to allow for some forward transfer (Ponti et al., 2022; Caccia et al., 2022). However, performance on previously seen tasks may decrease as we sequentially fine-tune on new ones (catastrophic forgetting). To account for both phenomenas, we include evaluation both right after *seeing a task* and right after *seeing the entire sequence* (Table 6).

Elastic Weight Consolidation (Kirkpatrick et al., 2017) We use the most validated method

		Baselines						Soft routing			
Task	Metric	Private		Shared		EWC		Sigmoid		Softmax	
QUOREF	QA-F1	71.15	±2.1	71.15	±2.1	71.15	±2.1	71.15	±2.1	71.15	±2.1
WIKI-SPLIT	Rouge-L	80.67	±0.3	80.83	±0.4	80.54	±0.3	80.69	±0.3	80.44	±0.0
ETHOS-DISABILITY	C-F1	84.50	±1.3	88.65	±4.0	85.11	±4.6	84.50	±1.3	87.53	±3.2
YELP-POLARITY	C-F1	96.57	±0.5	96.97	±0.3	96.55	±0.3	97.02	±0.2	96.98	±0.0
SUPERGLUE-RTE	Acc	85.68	±0.2	52.95	±8.0	54.87	±3.5	85.44	±0.6	85.56	±0.6
GLUE-COLA	M-Corr	44.34	±7.9	18.84	±21.6	15.75	±16.3	46.90	±2.8	47.87	±1.8
ETHOS-SEXUAL	C-F1	75.74	±7.4	73.66	±6.7	76.89	±0.9	85.47	±3.1	80.42	±2.7
BLIMP-SCOPE	Acc	100.00	±0.0	65.83	±27.4	100.00	±0.0	99.33	±0.6	100.00	±0.0
AI2-ARC	Acc	29.32	±0.2	25.20	±3.0	26.64	±1.0	29.88	±1.8	28.43	±2.2
AMAZON-POLARITY	C-F1	95.53	±0.4	89.93	±4.3	94.13	±0.9	95.97	±0.4	95.77	±0.4
RACE-HIGH	Acc	32.35	±0.9	25.51	±0.7	27.16	±1.1	32.96	±0.7	30.19	±0.6
BLIMP-LICENSOR	Acc	99.83	±0.3	99.83	±0.3	99.83	±0.3	100.00	±0.0	99.67	±0.6
TWEET-EVAL	C-F1	57.57	±3.6	57.12	±0.7	57.68	±1.9	56.63	±1.9	56.80	±3.9
BREAK-QDMR	EM	8.21	±0.4	5.08	±1.1	5.34	±0.6	7.00	±0.6	7.31	±0.6
CRAWL-DOMAIN	EM	67.21	±2.0	58.54	±2.7	61.09	±1.5	67.98	±1.0	67.62	±0.6
FREEBASE-QA	EM	2.48	±0.2	0.98	±0.3	1.57	±0.4	2.47	±0.1	2.46	±0.4
GLUE-QNLI	Acc	93.41	±0.6	54.50	±2.9	55.02	±2.1	93.81	±0.6	93.24	±1.0
HATEXPLAIN	C-F1	46.76	±7.0	50.16	±1.8	48.20	±1.0	53.66	±0.9	51.46	±1.7
AG-NEWS	C-F1	76.89	±10.2	67.30	±5.0	78.62	±10.8	79.08	±17.2	87.56	±0.4
CIRCA	C-F1	58.95	±10.3	20.66	±8.5	43.33	±13.0	59.28	±10.5	66.76	±7.0
Average	—	65.36	±1.1	55.18	±0.5	58.97	±1.1	66.46	±1.1	66.86	±0.4

Table 2: Results on test tasks in Crossfit benchmark. We evaluate 3 baselines and 2 Soft Routing methods. For each method, we run 3 experiments with different random seeds and report the average along with standard deviation of the metrics. Tasks are presented in the same order as they are learned during experiments.

to prevent catastrophic forgetting. Kirkpatrick et al. (2017) proposes Elastic Weight Consolidation (EWC), which penalises large update on parameters that are important for previous tasks by introducing a regularisation term to the loss function. Given model parameters θ, the loss for a new task $\mathcal{L}_B$, and the model weights after fine-tuning on a previous task A, θ^*_A, the new loss is:

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i(\theta_i - \theta^*_{A,i})^2 \quad (1)$$

where λ is a hyperparameter specific to this method and F is the Fisher information matrix.

The Fisher information matrix describes the importance of each parameter for previous tasks and it can be approximated based on first-order derivatives (Kirkpatrick et al., 2017; Pascanu and Bengio, 2013).

In our case, we only apply EWC with respect to the immediately previous task, noting that LoRA adapters are low-capacity methods and having a penalisation term which accounts for all previous tasks may be detrimental for performance on new tasks. We report performance after seeing each task and after seeing the entire sequence (Table 3).

3.2.2 Soft routing

We implement the family of methods described in Algorithm 1. Specifically, we experiment with different initialisations of the routing vector α and with different normalisition functions applied to this vector.

Softmax We apply a softmax function to α, so that all the values are non-negative and add up to 1.

$$\text{Softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (2)$$

Sigmoid We apply a sigmoid function to each dimension of α and then divide by the sum (L1 norm).

$$\sigma(x) = \frac{1}{1 + e^{-x_i}} \quad (3)$$

Gumbel Softmax (Jang et al., 2016) We add random noise to α. To each component, we add $n_i = -\log(-\log(u))$, where $u \sim \mathcal{U}(0, 1)$. We then apply Equation 2.

Gumbel Sigmoid Used as well in Ponti et al. (2022), it ensures stochasticity while allowing for differentiable sampling.

$$\hat{\alpha}_i = \sigma \left[\log \frac{\sigma(\alpha_i) u}{(1 - \sigma(\alpha_i))(1 - u)} \right] \quad (4)$$

with $u \sim \mathcal{U}(0, 1)$.

	$\lambda = 0.1$				$\lambda = 1$				$\lambda = 10$			
Task	Forward		Cat. Forgetting		Forward		Cat. Forgetting		Forward		Cat. Forgetting	
COLA	38.42	±6.1	3.87	±4.5	38.42	±6.1	3.67	±4.7	38.42	±6.1	12.46	±8.4
MRPC	74.10	±1.7	51.20	±20.2	74.23	±2.0	50.17	±19.8	68.42	±1.4	68.16	±5.4
QNLI	75.80	±1.0	59.26	±10.8	76.41	±1.7	59.74	±10.4	69.90	±1.8	61.52	±5.4
QQP	74.47	±1.8	66.34	±4.5	73.42	±1.1	65.53	±4.8	66.38	±2.3	63.43	±9.3
RTE	62.79	±1.7	55.42	±3.4	62.90	±1.3	56.48	±3.8	56.29	±2.0	58.04	±1.5
SST2	83.63	±1.1	60.14	±13.7	84.29	±1.5	58.47	±12.5	75.34	±7.6	71.43	±6.2
STSB	81.10	±1.5	81.10	±1.5	80.38	±1.4	80.38	±1.4	78.38	±1.7	78.38	±1.7
Average	70.04	±0.8	53.90	±3.2	70.01	±0.8	53.49	±3.1	64.73	±1.2	59.06	±2.0

Table 3: EWC performance on the GLUE dataset, for different λ values. We evaluate the model on each task right after it learns a specific task (*Forward*), and after training on the entire sequence (*Cat. Forgetting*). We report average and standard deviation of the metrics with ten different random seeds. We observe that there exists a trade-off between the *Forward* and *Cat. Forgetting* performance, depending on the value of λ. It is expected as λ acts as a regulariser and increasing it prevents us from overfitting on the current task, but at the same time it alleviates catastrophic forgetting.

Parameter	Data Set			
	Crossfit		GLUE	
Alpha LR				
0.05	70.85	±0.2	**71.56**	±0.7
0.1	**71.49**	±0.1	71.31	±0.8
0.5	70.85	±0.2	70.43	±0.7
Init strategy				
Zeros	**70.85**	±0.2	**71.56**	±0.7
Mild	70.48	±1.2	71.49	±0.8
Strong	**70.92**	±0.4	**71.59**	±0.8

Table 4: Mean Average Validation Score and the STD for different $AlphaLR$ and $InitStrategy$, which we introduce in our method. Our default settings are $AlphaLR = 0.05$ and $InitStrategy = zeros$. We run 3 experiments with different seeds for each setting.

3.3 Hyperparameters

We largely follow the hyperparameters from the AdaptersHub (Pfeiffer et al., 2020), a library based on HuggingFace, that describes successful GLUE experiments using LoRA. For each GLUE task, we train for 50 epochs with a batch size of 16 and an initial learning rate $5e-4$. For CrossFit test tasks, we set the batch size to 16 and the number of epochs to 30. Then, we search over the learning rate (LR) using the Private Baseline in the space $\{1e-2, 5e-3, 1e-3, 1e-4\}$. We choose the best LR for each task based on the validation metric and fix it. We report task-specific learning rates in Table B and use them for all experiments. We try different ranks for LoRA $\{4, 8, 16\}$, and we choose $r = 8$ for GLUE and $r = 16$ for CrossFit based on validation performance. For both datasets, we set LoRA's β parameter to 0.25. We also try different λ for EWC (Table 3) and choose $\lambda = 1$. We limit the length of input sequence to 128 for GLUE and 512 for CrossFit. For CrossFit, we limit the length of output sequence to 64 and use weight decay of 1e-2 following (Ponti et al., 2022). We use Adam (Kingma and Ba, 2014) as our optimizer and linear learning rate scheduler, which starts with the base learning rate and linearly decays it to 0 at the end of training. If not stated otherwise, we use the first task ordering from Appendix A. All our tables report the tasks by the order in which they were seen.

To validate our experimental setup, we check that our Private baseline matches or improves upon BART$_{large}$ (Lewis et al., 2020) on CrossFit (Ponti et al., 2022). We execute our experiments on 1x A100 80GB NVIDIA GPU. A single run takes approximately 10 minutes on GLUE and 5 hours on Crossfit.

We test higher learning rates specifically applied to an α vector that we introduce in the Soft routing method, as significant changes are required, e.g., to converge to a solution that concentrates the weight at the current task. We report the results for different $AlphaLR$ in Table 4 and choose 0.1 for CrossFit and 0.05 for GLUE. We also test different initialisations of α: 1) $zeros$ corresponding to zero vector, 2) $mild$ we put a prior on an alpha for the current task so that after normalisation the weight for the current task is equal to 0.75, and set rest alphas to 0.0 3) $strong$ same as $mild$, but the prior is equal to 0.9. For both datasets, we stick with $zeros$ for $InitStrategy$ because it is more straightforward than $strong$, and both work approximately the same (Table 4).

We limit the number of examples for each split $\in \{train/dev/test\}$ to $500/500/1000$ for GLUE and $500/500/15000$ in Crossfit. We aim to simulate a low-resource continual learning setup, which makes the results comparable across different tasks and also allows us to observe forward transfer. We would not have to rely on forward transfer if we had enough samples to learn the task. Therefore, it only makes sense to study forward transfer in a low-resource setting. For Crossfit tasks, we generate the output using greedy decoding during inference.

4 Results

We report the main results of our experiments in Table 1 for GLUE and Table 2 for Crossfit. For these tables, we choose one random permutation of tasks, and for each task and method, we report the average metric score across 3 runs with different random seeds. Furthermore, in Table 5, we present another comparison of described methods for 3 different orderings of the tasks. Higher scores indicate better performance across all metrics used in our experiments.

In both benchmarks, our Soft Routing approach, both Softmax and Sigmoid based, achieve the highest average scores and outperform Private, Shared, and EWC baselines. Furthermore, we see that among baselines, simply training a single LoRA per task (Private baseline) is the most effective and reliable method. Finally, we do not observe clear evidence that our base Soft Routing methods (Softmax and Sigmoid) benefit from adding stochasticity (G-Softmax and G-Sigmoid) in the form of Gumbel noise.

Out of baselines that enable forward transfer, i.e., Shared and EWC, we barely observe any evidence of shared knowledge, even if we evaluate the model on a task immediately after training on it. However, enormous catastrophic forgetting is noticeable when assessing our model after training on the entire task sequence. In Table 6, we report the results for GLUE, when Shared and EWC approaches are evaluated both after training on a task and in the end. To get a better understanding, we add to the comparison one extra column (Random), which corresponds to the model that uses a linear classifier head trained for the task with randomly initialised LoRA parameters. As shown in the table, the results of the Shared baseline evaluated in the end are in the similar ballpark as Random, which indicates a complete loss of knowledge about previous tasks.

Method	Permutation				
	1	2	3	All	
Private	65.36	64.29	65.30	64.98	±1.0
Shared	55.18	55.05	57.03	55.75	±1.9
EWC	58.97	49.76	57.30	55.34	±4.9
Softmax	66.86	63.20	64.95	65.00	±1.9
Sigmoid	66.46	64.02	66.60	**65.69**	**±1.6**
G-Softmax	65.89	64.73	65.15	65.26	±1.2
G-Sigmoid	65.14	64.01	65.61	64.92	±1.1

Table 5: Mean Average Test Score for three different random orderings (Appendix A) of Crossfit Test tasks. For each ordering we run three experiments with different random seeds and report the standard deviation.

In Table 5, we see that the order in which the model sees the tasks significantly affects the average test score, which stretches from **63.20** to even **66.86** for a different random permutation. It aligns with our intuitions, as depending on a task position, a model either solely relies on its training data for the first one or has implicit access to all previous tasks for the last one. To argue it further, in Figure 3, we plot the difference in test score between the Sigmoid Soft Routing method and the Private baseline against the task position for 3 different task orderings. Forward transfer rarely happens for the first few tasks as the latter tasks are more likely to benefit from previously trained parameters.

Our empirical results are promising, since we find Soft Routing approaches are superior to the baselines for different permutations of tasks (Table 5). However, the gap between Soft Routing and Private baseline is not significant (**0.02** for Softmax, and **0.71** for Sigmoid) We cannot conclude that one method is consistently better than the other, given the high standard deviation of the results (**1.9** and **1.6** for Softmax and Sigmoid, respectively).

5 Related Work

Parameter Efficient Fine-tuning To fine-tune large pre-trained models efficiently, some recent work introduces task-specific modules with fewer trainable parameters in model architecture and only updates these parameters while freezing the pre-trained weights. **Adapter** (Houlsby et al., 2019) introduces a bottleneck layer in each layer of Transformer model (Vaswani et al., 2017) as

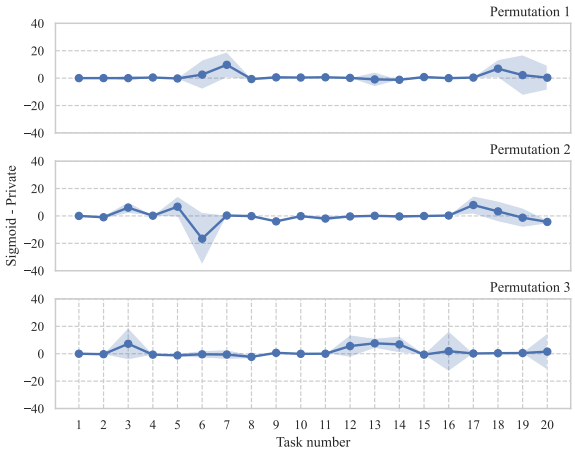


Figure 3: Difference in the test metric between the Soft Routing (Sigmoid) approach and the Private approach w.r.t. task number. We plot it for 3 different Cross-fit test tasks orderings (Appendix A). Whenever the difference is positive we speculate that this happens due to the Forward Transfer which is possible for the Soft Routing method. We see that it generally happens for latter tasks, which is consistent with our intuitions about Forward Transfer, as these can use parameters optimised for previous tasks to better generalise on new ones. We observe that tasks for which the difference is negative demonstrate high variance.

	Shared					
Task	Forward		Cat. Forgetting		Random	
COLA	38.42	±6.1	3.37	±4.3	0.06	±1.9
MRPC	74.65	±1.8	50.73	±21.0	68.24	±1.4
QNLI	75.71	±1.3	59.34	±9.3	62.87	±2.0
QQP	74.13	±1.8	68.45	±5.3	45.88	±1.7
RTE	62.43	±1.7	54.73	±3.6	52.13	±1.9
SST2	83.78	±1.2	53.58	±8.3	46.56	±3.6
STSB	80.72	±1.2	80.72	±1.3	52.08	±1.4
Average	69.90	±0.8	52.99	±2.9	46.83	±0.8

Table 6: Aggregate and task-specific results for GLUE test tasks. We evaluate the shared baseline in three different ways: after training on each individual task (*Forward*), after training on the entire sequence (*Cat. Forgetting*) and with using a random re-initialisation of the LoRA adapters and the fine-tuned classifier head (*Random*). For each method we run 10 experiments with different random seeds. Tasks are presented in the same order as they were learned during experiments.

learnable parameters. **Prefix fine-tuning** (Li and Liang, 2021) proposes to prepend task-specific prefix matrices to key and value vectors in multi-head attention blocks. **LoRA** (Hu et al., 2021) combines trainable low-rank matrices with pre-trained weights in multi-head attention. **UniPELT** (Mao et al., 2021) incorporates various parameter efficient methods in a united framework and select the appropriate modules for a specific task via gating mechanism.

Parameter efficient fine-tuning methods can learn modular and composable representations, i.e. trained parameters are isolated and can be combined together to share knowledge across multiple tasks, which are useful for multi-task learning and transfer learning. For example, **AdapterFusion** (Pfeiffer et al., 2021) is a two-stage method that efficiently promotes the transfer of knowledge across different tasks to solve a target task. They propose to train adapter modules for a set of tasks separately to encode task-specific knowledge and then combine these learned representations by an attention-based aggregation layer. **Polytropon** (Ponti et al., 2022) proposes to jointly learning a set of latent modular skills (implemented by sparse or low rank adapters) and a task-skill allocation matrix that activates a subset of relevant skills for a specific task.

Different from transfer learning and multitask learning settings, where the model does not suffer from catastrophic forgetting , we would like to achieve knowledge composition and avoid forgetting in the scenario of continual learning. As the model is trained sequentially on different tasks, we add task-specific adapters dynamically and leverage knowledge from previous tasks to learn the latest task. To the best of our knowledge, there has not been much prior work on designing parameter efficient methods for continual learning.

Continual Learning Existing work on continual learning can be classified into three categories. *Regularisation*-based approaches (Kirkpatrick et al., 2017; Zenke et al., 2017; Schwarz et al., 2018) estimate the importance of model parameters for previous tasks and introduce a regularisation term to loss function to prevent large update on important parameters. These approaches are prone to forgetting previous knowledge as the stream of tasks grow in size and diversity. (Van de Ven and Tolias, 2019) *Memory replay* (Shin et al., 2017; Rolnick et al., 2019; Sun et al., 2019) mitigates catastrophic forgetting by retraining the model on real or synthetic samples from previous tasks. These methods require a memory buffer to store data samples, which makes them inefficient for learning a long sequence of tasks. *Parameter isolation* methods (Rusu et al., 2016; Yoon et al., 2017; Mallya and Lazebnik, 2018; Schwarz et al., 2018; Wortsman et al., 2020) al-

locate dedicated parameters for each task either by dynamically expanding the network with new modules or pruning. Our approach follows the line of work on developing dynamic architecture with isolated task-specific parameters. However, it is more parameter efficient and scales well at handling long stream of tasks. Besides, our modular design provides a simple and explicit mechanism to promote knowledge transfer.

6 Conclusions and Future Work

We propose a new method for a continual learning setting based on soft routing, called Modular Continual Learning, which encourages forward transfer and prevents catastrophic forgetting. We experiment with different methods for soft routing (Softmax, Sigmoid, Gumbel Noise), different initialisations, and optimisation strategies. We compare it to three strong baselines, Private Adapter, Shared Adapter, and EWC. We obtain promising results in preliminary experiments, however, we can not claim the superiority of our method due to insignificant improvement and poor robustness.

Despite our best effort to ensure the fairest possible comparison between approaches within the limited timeline of this project, we must state that the hyperparameter search may be favoring the private baseline. We choose the batch size and learning rate for each task based on the validation score of the private adapter to ensure our results are better than those of Ponti et al. (2022). However, this experimental design may put other methods at a disadvantage. In future work, we will overcome this by doing an internal hyperparameter search for each task conditioned on the performance on previous one. We propose to choose our hyperparameters from internal search based on three runs for three different permutations, one for each. This will somehow find the optimal parameters that can generalise to different permutations. However, given the model is sequentially trained on a stream of tasks, it is not clear whether this hyperparameter search strategy will produce better results on average for all the tasks.

From a technical perspective, if we are given some pre-trained adapters (soft routing) or a good initialisation (shared baseline), it is not only natural to do hyperparameter search on a new task, but also this has the same complexity as doing it for the private baseline. One obvious limitation of this will be that we may not be able to generalise these hyperparameters to different task orderings.

There are other experiments that we want to run in future work. Observing that the improvement of our method with respect to the private baseline is somewhat limited, we would like to see the performance of soft routing in experiments with less training data. Conversely, we would like to see if using more training data improves our current result or if the performance of a LoRA adapter will saturate. It is important to keep in hindsight that our method is not different from a LoRA adapter in modeling capacity; therefore, using different regimes of data will tell us the true potential of our method.

The same way data affects the performance of LoRA, the rank r is also expected to have some impact. It has been previously suggested that there may be an optimal rank for certain tasks. Hu et al. (2021) shows that in the MultiNLI dataset (Williams et al., 2018) no improvement can be achieved above $r = 2$. Therefore, we will reduce the rank of LoRA, with the hypothesis that our approach could be better if we limit the capacity for private approach.

There are some slight modifications of our method that could be proven useful. One interesting baseline will be to observe forward transfer by using soft routing on top of private adapters, for which we could use either CrossFit train or test tasks; that is, given $(n-1)$ previous private adapters, we use soft routing on top of them to learn the n-th task. Another baseline would be initialising LoRAs for a new task as a standard average of LoRAs trained for some training tasks, following (Caccia et al., 2022).

We will also experiment about changing the model for the CrossFit experiments to BART, following related work that also evaluates on CrossFit test tasks (Ponti et al., 2022; Caccia et al., 2022) and reports more potential for improvement than in our current setup with T5, which may already be saturated. We will also explore extending our method to other adapters; for instance, Mahabadi et al. (2021) explored using a GeLU nonlinearity Hendrycks and Gimpel (2016) after applying the first projection matrix A. We hypothesise that adding extra expressivity in the form of non-linearity can allow us to better incorporate the knowledge stored in parameters learned from previous tasks.

7 Contributions

Guillem collaborated in our initial Github codebase and with the initial GLUE experiments. He implemented the CrossFit integration of our experiments. He integrated EWC to our codebase. He executed experiments on both GLUE and Crossfit. He implemented the Soft Routing approach in collaboration with Piotr.

He sketched and designed Figure 1, 2 and did an initial literature review.

He also wrote Section 2 (Method), Subsection 3.2 (Models and baselines), Section 6 (Conclusions and Future Work). He ran the experiments that formed Table 1, 3, 6. He ran the experiments of Table 4 corresponding with the GLUE experiments.

Christina contributed to the implementation of EWC baseline and conducted experiments on GLUE. She designed visual illustrations of our method. She drafted the Abstract, Introduction, Datasets and Related Work sections in the report.

Piotr created our initial GitHub codebase that supports running GLUE experiments on the external cluster. He executed experiments on both GLUE and Crossfit. He implemented the Soft Routing approach in collaboration with Guillem. He wrote the Hyperparameters and Results section in the report. He created the Appendix A, Appendix B, Table 5, Table 4, Table 2, Figure 3.

References

Lucas Caccia, Edoardo Ponti, Lucas Liu, Matheus Pereira, Nicolas Le Roux, and Alessandro Sordoni. 2022. Multi-head adapter routing for data-efficient fine-tuning.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding.

Robert French. 1999. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3:128–135.

Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus).

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax.

Zixuan Ke, Bing Liu, Nianzu Ma, Hu Xu, and Lei Shu. 2021. Achieving forgetting prevention and knowledge transfer in continual learning. *Advances in Neural Information Processing Systems*, 34:22443–22456.

Diederik P. Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Sungjin Lee. 2017. Toward continual learning for conversational agents. *arXiv preprint arXiv:1712.09943*.

Matthias Lenga, Heinrich Schulz, and Axel Saalbach. 2020. Continual learning for domain adaptation in chest x-ray classification. In *Medical Imaging with Deep Learning*, pages 413–423. PMLR.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for

generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online. Association for Computational Linguistics.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *arXiv preprint arXiv:2205.05638*.

Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. 2021. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks.

Arun Mallya and Svetlana Lazebnik. 2018. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 7765–7773.

Yuning Mao, Lambert Mathias, Rui Hou, Amjad Almahairi, Hao Ma, Jiawei Han, Wen-tau Yih, and Madian Khabsa. 2021. Unipelt: A unified framework for parameter-efficient language model tuning. *arXiv preprint arXiv:2110.07577*.

Sahisnu Mazumder, Bing Liu, Shuai Wang, and Nianzu Ma. 2019. Lifelong and interactive learning of factual knowledge in dialogues. *arXiv preprint arXiv:1907.13295*.

Razvan Pascanu and Yoshua Bengio. 2013. Revisiting natural gradient for deep networks.

Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2021. AdapterFusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 487–503, Online. Association for Computational Linguistics.

Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. 2020. Adapterhub: A framework for adapting transformers.

Edoardo M. Ponti, Alessandro Sordoni, Yoshua Bengio, and Siva Reddy. 2022. Combining modular skills in multitask learning.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer.

Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. 2017. Learning multiple visual domains with residual adapters.

David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. 2019. Experience replay for continual learning. *Advances in Neural Information Processing Systems*, 32.

Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. 2016. Progressive neural networks. *arXiv preprint arXiv:1606.04671*.

Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. 2018. Progress & compress: A scalable framework for continual learning. In *International Conference on Machine Learning*, pages 4528–4537. PMLR.

Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. 2017. Continual learning with deep generative replay. *Advances in neural information processing systems*, 30.

Fan-Keng Sun, Cheng-Hao Ho, and Hung-Yi Lee. 2019. Lamol: Language modeling for lifelong language learning. *arXiv preprint arXiv:1909.03329*.

Marcos Treviso, Tianchu Ji, Ji-Ung Lee, Betty van Aken, Qingqing Cao, Manuel R Ciosici, Michael Hassid, Kenneth Heafield, Sara Hooker, Pedro H Martins, et al. 2022. Efficient methods for natural language processing: a survey. *arXiv preprint arXiv:2209.00099*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.

Gido M Van de Ven and Andreas S Tolias. 2019. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding.

Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122. Association for Computational Linguistics.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2019. Huggingface's transformers: State-of-the-art natural language processing.

Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. 2020. Supermasks in superposition. *Advances in Neural Information Processing Systems*, 33:15173–15184.

Qinyuan Ye, Bill Yuchen Lin, and Xiang Ren. 2021. Crossfit: A few-shot learning challenge for cross-task generalization in nlp.

Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. 2017. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*.

Fajie Yuan, Guoxiao Zhang, Alexandros Karatzoglou, Joemon Jose, Beibei Kong, and Yudong Li. 2021. One person, one model, one world: Learning continual user representation without forgetting. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 696–705.

Friedemann Zenke, Ben Poole, and Surya Ganguli. 2017. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995. PMLR.

Appendix

A Task orderings

Ordering		
1	2	3
QUOREF	GLUE-COLA	AI2-ARC
WIKI-SPLIT	BREAK-QDMR	WIKI-SPLIT
ETHOS-DISABILITY	ETHOS-DISABILITY	CIRCA
YELP-POLARITY	BLIMP-LICENSOR	QUOREF
SUPERGLUE-RTE	CIRCA	RACE-HIGH
GLUE-COLA	AG-NEWS	CRAWL-DOMAIN
ETHOS-SEXUAL	WIKI-SPLIT	TWEET-EVAL
BLIMP-SCOPE	AMAZON-POLARITY	SUPERGLUE-RTE
AI2-ARC	SUPERGLUE-RTE	GLUE-QNLI
AMAZON-POLARITY	BLIMP-SCOPE	FREEBASE-QA
RACE-HIGH	QUOREF	BLIMP-SCOPE
BLIMP-LICENSOR	CRAWL-DOMAIN	ETHOS-SEXUAL
TWEET-EVAL	GLUE-QNLI	ETHOS-DISABILITY
BREAK-QDMR	RACE-HIGH	HATEXPLAIN
CRAWL-DOMAIN	FREEBASE-QA	BREAK-QDMR
FREEBASE-QA	YELP-POLARITY	AG-NEWS
GLUE-QNLI	ETHOS-SEXUAL	BLIMP-LICENSOR
HATEXPLAIN	HATEXPLAIN	YELP-POLARITY
AG-NEWS	TWEET-EVAL	AMAZON-POLARITY
CIRCA	AI2-ARC	GLUE-COLA

Table 7: Different Crossfit Test tasks orderings (randomly chosen) we use in our experiments.

B Crossfit statistics

Task	Train/Dev/Test samples	STD	Learning Rate
QUOREF	181/160/12090	2.1	0.001
WIKI-SPLIT	160/160/25000	0.3	0.005
ETHOS-DISABILITY	160/160/435	1.3	0.005
YELP-POLARITY	160/160/38000	0.5	0.001
SUPERGLUE-RTE	160/160/1385	0.2	0.0001
GLUE-COLA	160/160/5215	7.9	0.005
ETHOS-SEXUAL	160/160/435	7.4	0.005
BLIMP-SCOPE	160/160/1000	0.0	0.005
AI2-ARC	160/160/1495	0.2	0.005
AMAZON-POLARITY	160/160/5000	0.4	0.005
RACE-HIGH	160/160/17255	0.9	0.001
BLIMP-LICENSOR	160/160/1000	0.3	0.005
TWEET-EVAL	160/160/4775	3.6	0.001
BREAK-QDMR	160/160/38800	0.4	0.005
CRAWL-DOMAIN	160/160/9765	2.0	0.001
FREEBASE-QA	160/160/19970	0.2	0.005
GLUE-QNLI	160/160/27315	0.6	0.001
HATEXPLAIN	240/240/9610	7.0	0.005
AG-NEWS	320/320/38000	10.2	0.005
CIRCA	400/400/33500	10.3	0.001

Table 8: Crossfit Test tasks statistics. We report the number of samples per split $\in \{Train, Dev, Test\}$ along with the standard deviation of the Private Baseline trained on each task independently. We do not observe any relation between the number of samples available for the task and the variance of our results. We also report the task-specific Learning Rates that we used in all our Crossfit experiments. We choose it among $\{5e-2, 1e-2, 5e-3, 1e-3\}$ based on the validation metric.